OpenJDK 11 · WIT vs Direct · atualizado 16/10/2026

Dossiê dos experimentos

Tudo que foi executado, com hipótese, método, resultado e ressalva.

PARTE I — INTRODUÇÃO RESUMIDA

Em três minutos

Se entregarmos a um LLM **as precondições que fazem cada método lançar exceção**, ele gera testes melhores para o contrato excepcional?

Resposta curta: sim em direcionamento e verificação, não em cobertura de código. E o ganho só se materializa quando o teste tem como reprovar — o que depende do prompt, não do contexto.

Foram **cinco experimentos de geração** (um principal, três intervenções e um controle negativo) e **três verificações de infraestrutura**. Ao longo do caminho foram encontrados e corrigidos **quatro defeitos de medição**, todos com o mesmo padrão: falhavam parecendo sucesso.

MIRA NA EXCEÇÃO
1,30–1,62×
nas 4 condições, inclusive no controle

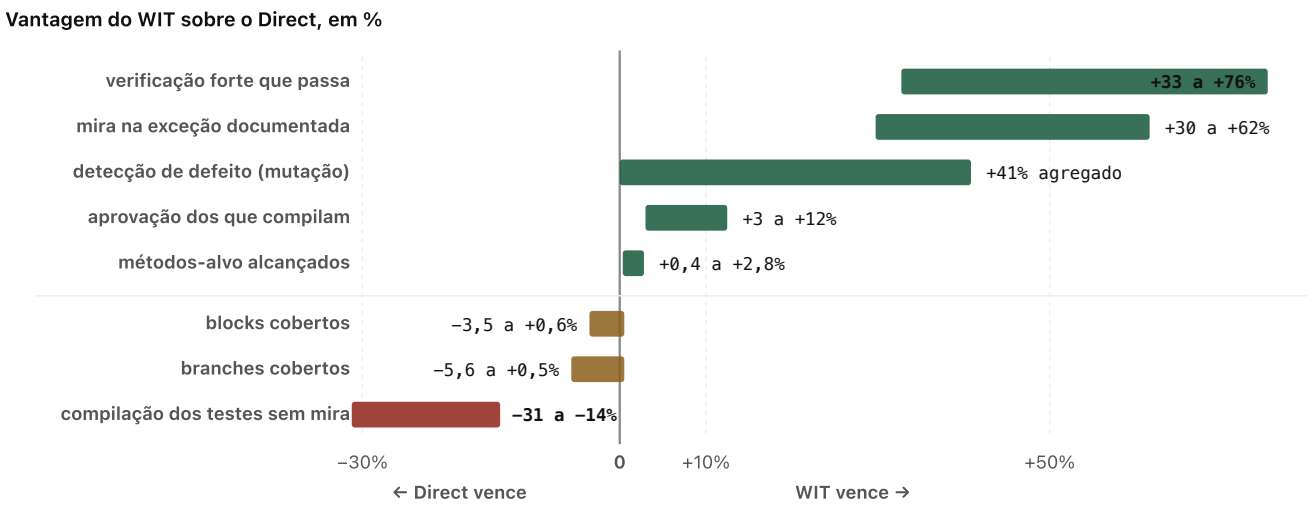
VERIFICAÇÃO QUE PASSA
1,33–1,76×
nas 3 intervenções · 1,08× no controle

DETECÇÃO DE DEFEITO
1,41×
mortes exclusivas, 817 sites pareados

COBERTURA DE BRANCHES
0,94–1,01×
Direct vence em 4 de 5 condições

O balanço, numa escala só

Cada barra é a faixa observada entre as condições experimentais. À direita do zero, o WIT vence; à esquerda, o Direct.



As cinco barras de cima são favoráveis ao WIT; as três de baixo, desfavoráveis. A assimetria de magnitude importa: as vantagens são de dezenas de pontos percentuais, as desvantagens em cobertura são de poucos pontos — mas a última barra, a compilação dos testes que não miram, é uma desvantagem grande e consistente.

A favor do WIT

1 · Direciona o teste à exceção documentada — 1,30–1,62× mais testes capturam o tipo certo. É o resultado mais robusto: vale nas quatro condições, **inclusive no controle negativo**, o que mostra que vem do contexto e não do prompt.

O mesmo método, os dois braços. O Direct nunca passa `null` porque não sabe que essa condição existe:

DIRECT · `Arrays.setAll`

```
double[] array = new double[5];
IntToDoubleFunction gen = i -> i * 2.0;
Arrays.setAll(array, gen);

for (int i = 0; i < array.length; i++) {
    if (array[i] != i * 2.0) {
        throw new AssertionError(
            "Unexpected value at " + i);
    }
}
```

WIT · o mesmo arquivo

```
double[] array = {1.0, 2.0, 3.0};
try {
    Arrays.setAll(array, null);
    throw new AssertionError(
      "Expected NPE was not thrown");
} catch (NullPointerException e) {
    // esperado
} catch (Throwable t) {
    throw new AssertionError(
      "Unexpected type: " + t);
}
```

2 · Essa mira converte em verificação útil — quando o oráculo tem como reprovar, 1,33–1,76× mais testes verificam o contrato e passam. Em números absolutos, no melhor caso são 1.195 testes úteis contra 681: **514 a mais, sem custo adicional de geração**.

3 · Detecta ao menos tantos defeitos quanto o Direct — mutação dirigida em 817 sites pareados: 89 mortes exclusivas do WIT contra 63 do Direct. Em nenhuma das cinco condições o Direct supera.

4 · Os testes do WIT compilam e passam um pouco mais — a taxa de aprovação entre os que compilam é 3 a 12 pontos percentuais maior, nas quatro condições.

Contra o WIT

1 · Cobre menos código dentro de cada método — o Direct vence em blocks e branches em quatro das cinco condições. A causa está **no mesmo exemplo acima**: o teste Direct percorre um laço de cinco elementos e verifica cinco valores; o teste WIT faz **uma** chamada e para.

Um caminho de exceção é curto por natureza. Passa `null`, confirma que a exceção certa veio, encerra. Isso é uma propriedade do que se está testando, não um defeito do método — mas significa que **apresentar cobertura como métrica principal desfavorece o WIT**.

2 · Quando o WIT erra o alvo, erra pior — este é o achado desfavorável mais consistente. Nos testes que *não* miram exceção nenhuma, o WIT compila de 7 a 11 pontos percentuais **abaixo** do Direct, nas quatro condições:

CONDIÇÃO	DIRECT COMPILA	WIT COMPILA	DIFERENÇA
V1 instrução	35,8%	24,6%	**−11,2pp**
V2 modelo mini	46,2%	39,8%	**−6,4pp**
V4 few-shot	38,5%	28,1%	**−10,4pp**
V3a controle	35,5%	26,6%	**−8,9pp**

Um exemplo real, do braço WIT, que não compilou. O teste é logicamente bem formado e tem oráculo forte — mas usa uma classe que não é pública:

`Normalizer2WitupTest_P65663701f6d0.java` · V1, braço WIT

```
import sun.text.normalizer.Normalizer2;

public class Normalizer2WitupTest_P65663701f6d0 {
    public static void main(String[] args) { testGetNFKDInstance(); }

    public static void testGetNFKDInstance() {
        try {
            Normalizer2 result = Normalizer2.getNFKDInstance();
            if (result == null) {
                throw new AssertionError("FAIL: getNFKDInstance() returned null");
            }
        } catch (Throwable t) {
            throw new AssertionError("FAIL: ... threw an exception: " + t);
        }
    }
}
```

`error: Normalizer2 is not public in sun.text.normalizer; cannot be accessed from outside package` — o modelo inventou um acesso que não existe. Quando o contexto WIT não leva a uma condição de exceção construível, o que sobra tende a ser mais frágil que o palpite genérico do Direct.

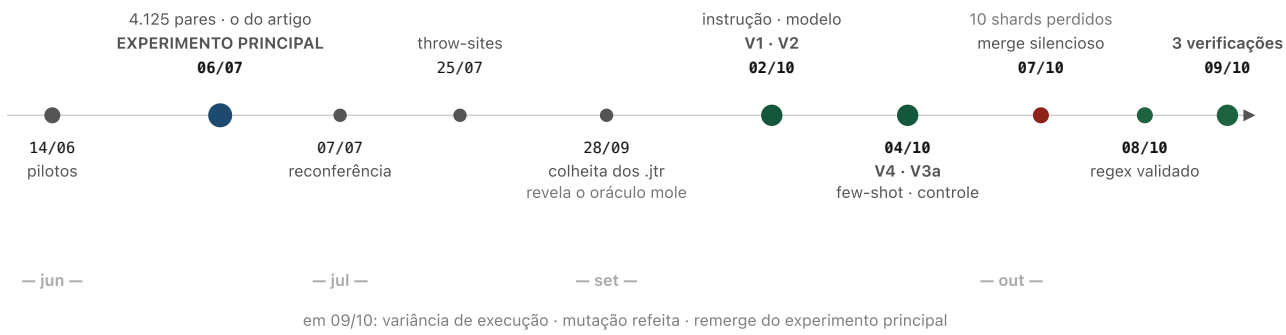
3 · Todo o ganho depende do oráculo ter voz — no controle de temperatura, onde 59,5% dos testes mirados ficam sem mecanismo de reprovação, a vantagem cai de 1,33–1,76× para **1,075×**. O contexto sozinho não entrega nada: ele precisa de um prompt que exija o oráculo.

4 · A taxa de compilação é baixa nos dois braços — entre 42% e 56%. É limitação do método como um todo, não do WIT, mas significa que aproximadamente metade do orçamento de geração é desperdiçada.

As três ressalvas honestas

- **Sem barra de erro.** As réplicas V3b e V3c nunca foram geradas — pararam no limite de cobrança da OpenAI. Não se sabe quanto da diferença entre 1,33× e 1,76× é efeito e quanto é ruído de amostragem.
- **A especificação vem do próprio código.** O WIT lê o código para descobrir qual exceção é lançada; o teste então confere isso. Se o código tivesse um bug, o teste confirmaria o bug. Mede **conformidade de contrato**, não detecção de defeito real.
- **O operador de mutação é grosseiro.** Remover o `athrow` faz o método retornar em vez de lançar — qualquer teste com voz que alcance o site detecta. Ele mede alcance mais que precisão do oráculo.

Linha do tempo



Azul: o experimento que gerou o artigo. Verde-escuro: as quatro variantes de geração. Vermelho: defeito de infraestrutura descoberto. Verde: verificação concluída. O intervalo de julho a setembro corresponde à mineração de resultados sobre os dados já coletados.

O fio condutor

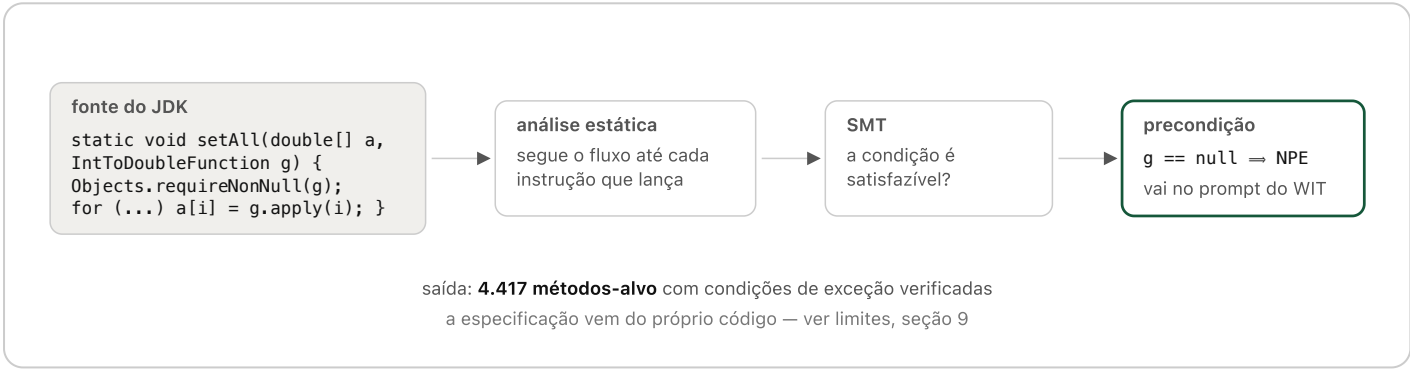
Em **julho** o experimento principal mostrou algo incômodo: o WIT vencia em aprovação de testes, mas empatava em verificação de exceção. Em **setembro**, ao colher os arquivos de resultado individuais, apareceu a causa — boa parte dos testes **mirava a exceção certa e não conseguia reprovar**. Em **outubro** foram geradas quatro variantes para testar se esse "oráculo mole" era artefato do prompt. Era. E corrigi-lo revelou uma vantagem do WIT que estava escondida.

PARTE II — DETALHAMENTO

1 · O problema

Peça a um modelo "escreva testes para `Arrays.setAll`" e ele escreve testes do caminho normal: argumentos válidos, confere o resultado. É útil, mas deixa de fora o que a documentação promete sobre erro.

O JDK documenta, para milhares de métodos, qual exceção é lançada e sob qual condição. É especificação escrita, verificável — e normalmente ignorada na geração de testes. O **contexto WIT** extrai essas condições por análise estática, verifica cada uma com um resolvedor SMT para descartar as insatisfazíveis, e as entrega ao modelo.



---

2 · Os conceitos, com código real

Todos os exemplos são testes **efetivamente gerados** no experimento.

2.1 · O oráculo e a regra implícita do jtreg

Oráculo é o critério que decide se um teste aprova ou reprova. Sob o jtreg, um teste é um `public static void main` e a regra é uma só:

```
APROVA  se main() retorna normalmente
REPROVA se main() lança qualquer exceção
```

Disso decorre a consequência central deste trabalho: **imprimir uma mensagem de falha não reprova o teste**.

2.2 · Mira — o teste aponta para a exceção certa?

É o eixo que o contexto WIT influencia: ele diz ao modelo *onde* está a exceção. Mesmo método, mesmo arquivo, braços diferentes:

SEM MIRA · braço Direct

```
double[] array = new double[5];
IntToDoubleFunction gen = i -> i * 2.0;
Arrays.setAll(array, gen);

for (int i = 0; i < array.length; i++) {
    if (array[i] != i * 2.0) {
        throw new AssertionError(
            "Unexpected value at " + i);
    }
}
```

Testa o caminho normal. Nunca passa `null` — não sabe que a condição existe.

COM MIRA · braço WIT

```
double[] array = {1.0, 2.0, 3.0};
try {
    Arrays.setAll(array, null);
    throw new AssertionError(
      "Expected NPE was not thrown");
} catch (NullPointerException e) {
    // esperado
} catch (Throwable t) {
    throw new AssertionError(
      "Unexpected type: " + t);
}
```

Vai à condição documentada e exige o tipo exato — o `catch (Throwable)` rejeita a exceção errada.

Repare na troca: o Direct percorre um laço de cinco elementos; o WIT faz **uma** chamada e para. Mais verificação de contrato, menos cobertura de código — no mesmo método. É a explicação mecânica do resultado 3.

2.3 · Voz — o teste consegue reprovar?

Sem voz, o teste executa o código, observa o erro e **aprova mesmo assim**:

`ArrayListWitupTest_P43cd15ea8027.java` · braço WIT · classificado **FRÁGIL**

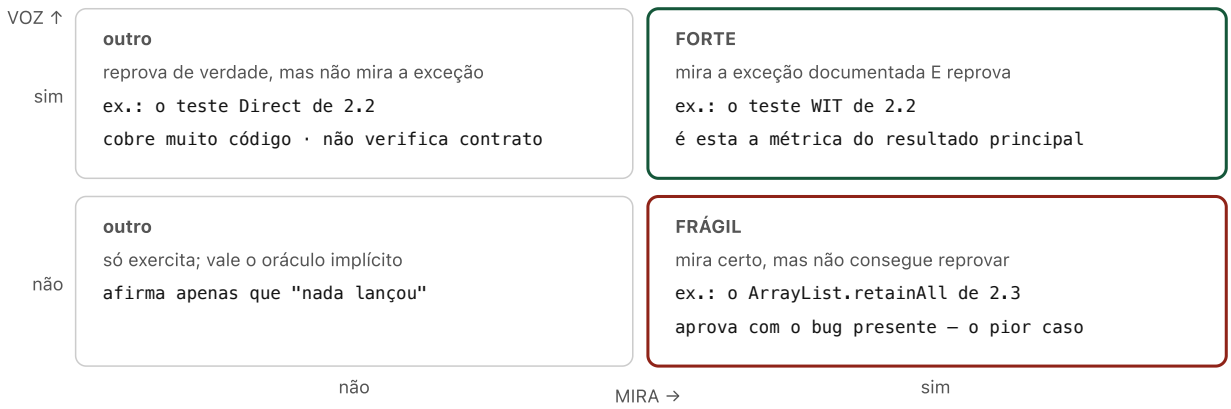
```
ArrayList<Object> list = new ArrayList<>();
list.add("element");
try {
    list.retainAll(null);
    System.out.println("FAILED: Expected NullPointerException not thrown.");
} catch (NullPointerException e) {
    System.out.println("PASSED: NullPointerException thrown as expected.");
}
```

A mira está certa. Mas se `retainAll(null)` deixar de lançar — o bug que queremos pegar — o teste imprime `FAILED:` e **retorna normalmente**. O jtreg aprova. O defeito passa.

Mecanismos que contam como voz, porque fazem `main` lançar ou encerrar com código de erro:

```
throw new AssertionError(...)     throw new RuntimeException(...)
System.exit(1)                    Asserts.fail(...)
assert cond : "msg"               ← vale: -ea ligado em todas as execuções
```

2.4 · A classificação



A classificação é feita sobre o **código-fonte** de cada teste, então cobre 100% dos gerados — inclusive os que não compilam. É por isso que o resultado principal sobreviveu aos defeitos de infraestrutura: ele não depende de execução nem de cobertura.

3 · Os experimentos, um a um

E0 Experimento principal pareado 06/07/2026

HIPÓTESE	O contexto WIT melhora os testes gerados para métodos com exceção documentada.
MÉTODO	4.125 pares de testes (direct × wit) para os mesmos métodos, mesmo modelo, mesma temperatura. 48 shards no CodeBuild com jtreg + JCov.
RESULTADO	WIT vence aprovação de testes (38,96% contra 35,27%) e uso de `try/catch` (95,3% contra 72,9%). Direct vence cobertura estrutural. **Verificação de exceção: empate.**
PROBLEMA	O empate em verificação não era esperado. A causa só apareceu em setembro.

– Colheita dos arquivos de resultado individuais 28/09/2026

MOTIVO	Investigar por que a verificação empatava.
ACHADO	**O oráculo mole.** Boa parte dos testes do braço WIT mirava a exceção certa e **não tinha mecanismo para reprovar** — imprimia a falha e o harness aprovava. A vantagem existia, mas não conseguia se expressar.
PERGUNTA ABERTA	Isso é propriedade do método ou artefato do prompt?

As quatro variantes abaixo nasceram dessa pergunta. Cada uma muda **exatamente uma** coisa no prompt-base, e três atacam explicações diferentes para o oráculo mole.

V1 Instrução explícita de oráculo 02/10/2026

HIPÓTESE	O oráculo é mole porque **ninguém disse ao modelo** que imprimir não reprova.
INTERVENÇÃO	Acrescenta ao prompt: *"o teste DEVE sinalizar a falha ao harness — lance uma exceção ou chame System.exit(1). Apenas imprimir System.out.println("FAIL: ...") NÃO reprova o teste."*
RESULTADO	Voz em **99,8%** dos testes mirados. Verificação forte que passa: **1,755×** (681 → 1.195). A maior das quatro.

V2 Modelo mais capaz 02/10/2026

HIPÓTESE	O oráculo é mole por **limitação de capacidade** do modelo.
INTERVENÇÃO	Prompt-base inalterado. Troca `gpt-4.1-nano` por `gpt-4.1-mini`. É a única variante em que o modelo muda.
RESULTADO	Voz em **99,9%** sem que ninguém pedisse. Verificação: **1,329×** (940 → 1.249) — a menor das três intervenções, porque o braço Direct também melhora.

V4 Exemplo no cabeçalho (few-shot) 04/10/2026

HIPÓTESE	Demonstrar vale mais que descrever.
INTERVENÇÃO	Acrescenta um modelo de cabeçalho obrigatório que **contém** um `throw new AssertionError("expected NPE")`.
RESULTADO	Voz em **97,4%**. Verificação: **1,705×** (655 → 1.117), praticamente empatada com a instrução explícita.
RESSALVA	O exemplo foi escrito para fixar o **formato** do cabeçalho, mas inevitavelmente demonstra também um oráculo forte. Não é um teste limpo de "demonstrar vs descrever".

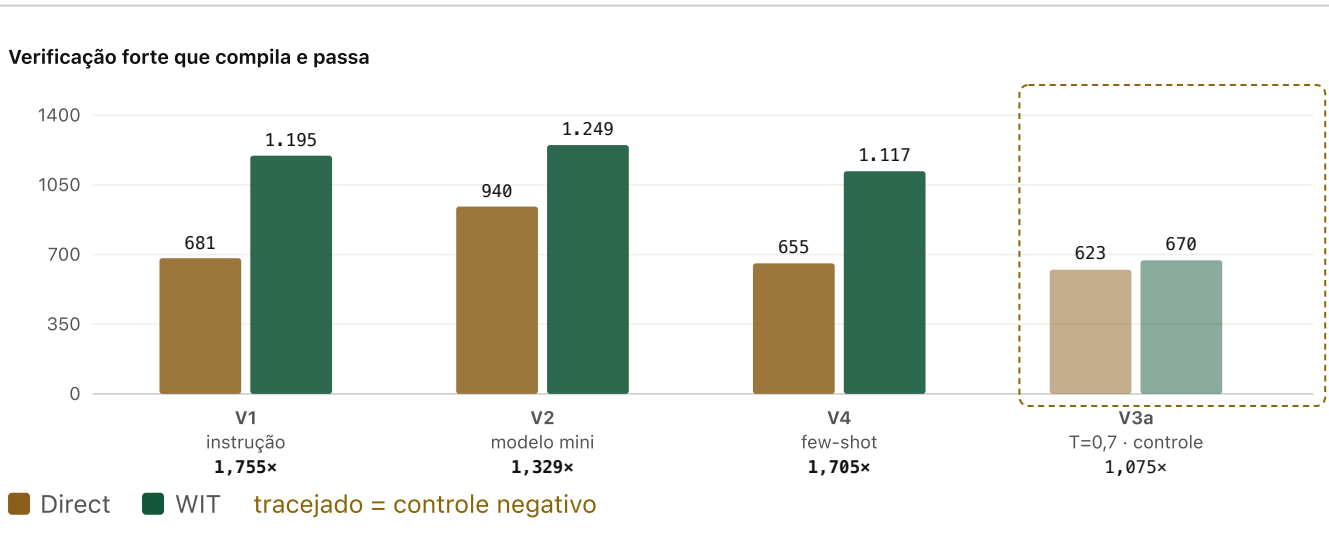
V3a Temperatura 0,7 — controle negativo 04/10/2026

HIPÓTESE	**Nenhuma.** Aumentar a aleatoriedade da amostragem não tem mecanismo plausível para ensinar o modelo a reprovar um teste.
PAPEL	Se produzisse o mesmo ganho das três intervenções, o efeito todo seria artefato de medição.
RESULTADO	Voz desaba para **59,5%**. Verificação: **1,075×** — fora da faixa das intervenções. **O controle cumpriu seu papel.**
PENDÊNCIA	As réplicas V3b e V3c, que dariam a barra de erro, nunca foram geradas — pararam no limite de cobrança da OpenAI com zero testes.

---

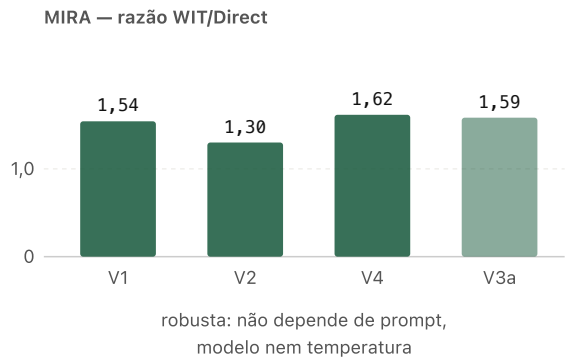
4 · Resultados consolidados

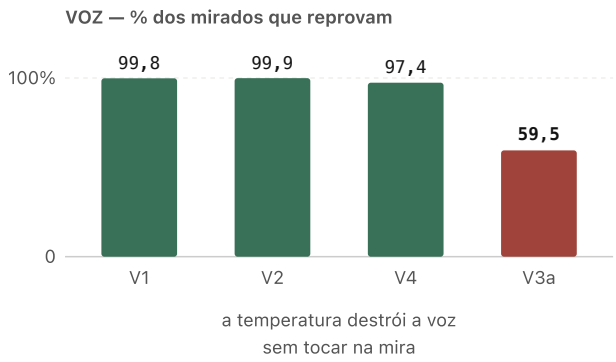
4.1 · O resultado principal



4.2 · Por que o controle é a peça decisiva

Os dois eixos do oráculo se comportam de forma diferente sob temperatura — e é isso que separa causa de coincidência.

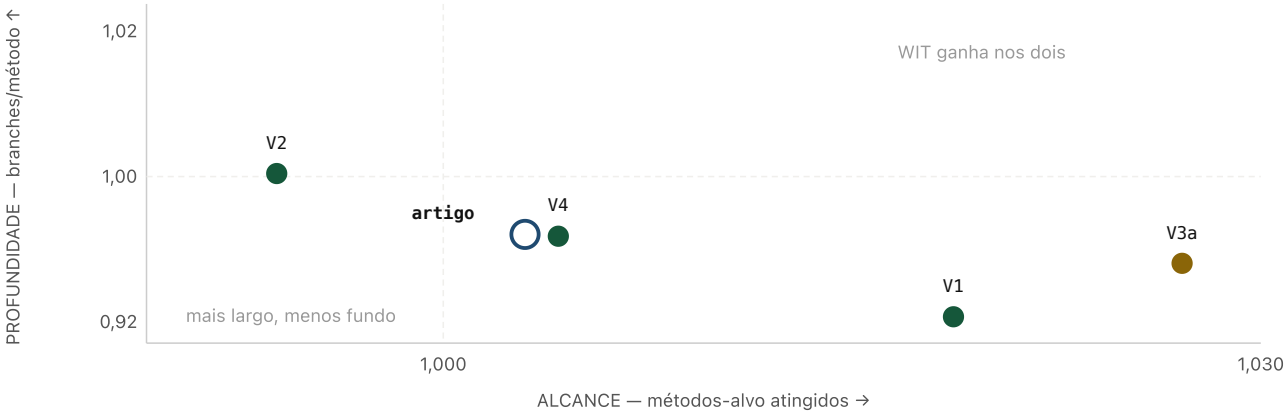




Elevar a temperatura derruba a voz de ~99% para 59,5% e deixa a mira praticamente intacta (1,59×). Como o **ganho final desaba junto com a voz, e não com a mira**, fica demonstrado que os dois mecanismos são necessários: contexto para mirar, oráculo com voz para reprovar.

4.3 · Alcance contra profundidade

Razão WIT/Direct, cobertura restrita aos métodos-alvo com denominador idêntico nos três braços. O ponto vazado é o experimento principal de julho, remesclado em 09/10.

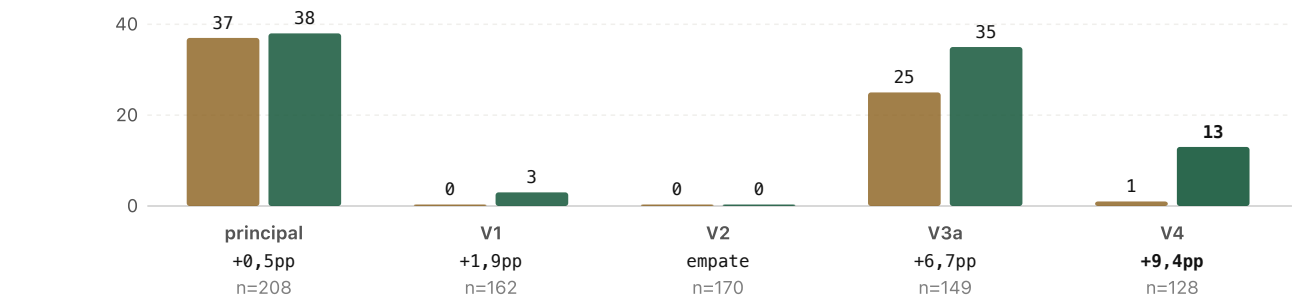


Os cinco pontos ficam à direita da vertical — o WIT alcança mais métodos-alvo em **todas** as condições. Quatro ficam abaixo da horizontal: menos profundidade. O experimento do artigo cai no meio do padrão das variantes, o que mostra que não é um caso à parte.

4.4 · Detecção de defeito — mutação dirigida

Removemos por ASM o `athrow` de um site de exceção e contamos como **morte** o teste que passa no código original e reprova no mutante. A taxa só é comparável nos sites em que **os dois braços** têm veredito válido.

Mortes exclusivas: sites que só um braço detecta



Agregado: 89 mortes exclusivas do WIT contra 63 do Direct em 817 sites pareados — razão 1,41×. Em nenhuma das cinco condições o Direct supera o WIT.

Onde ser cauteloso com este resultado

V1 e V2 estão no **teto**: 81% a 89% de mortes nos dois braços, com discordância de 0/3 e 0/0. Não há espaço para discriminar. O sinal vem de duas condições, não de cinco.

E o operador é grosseiro: removido o `athrow`, o método retorna em vez de lançar, e **qualquer** teste com voz que alcance o site detecta. Um operador que lance uma exceção de **tipo diferente** discriminaria melhor, porque só morreria para quem verifica o tipo — que é exatamente a mira.

---

5 · As verificações de infraestrutura

C1 Validação do classificador `08/10/2026`

RISCO A classificação forte/frágil depende de um padrão reconhecer mecanismos de voz. Se perdesse formas legítimas de reprovar, inflaria a contagem de frágeis — e é ela que sustenta a leitura do controle negativo.

MÉTODO Reexecutar a classificação com um padrão deliberadamente mais permissivo e ver se as razões se movem.

RESULTADO Dos 1.570 frágeis examinados, apenas **1,1%** tinham mecanismo não reconhecido. V1, V2 e V4 ficam **idênticos**; V3a vai de 1,075× a 1,077× — um teste. **As conclusões não dependem da definição do padrão.**

C2 Variância de execução 09/10/2026

RISCO O jtreg roda com `-timeout:4` e concorrência. Teste sensível a tempo pode mudar de resultado entre execuções, o que tornaria as taxas de aprovação irreprodutíveis.

MÉTODO Reexecutar 8 shards num prefixo separado do S3, preservando o original, e comparar os arquivos de resultado **teste a teste**.

RESULTADO **726 testes em comum, 726 com o mesmo desfecho, zero mudanças.** Escopo: 8,7% dos testes daquela condição. A afirmação defensável é *"não foi observada instabilidade em 726 testes"*, não *"não existe instabilidade"*.

C3 Remerge do experimento principal 09/10/2026

RISCO O merge que produziu os números publicados usava uma opção que **descarta arquivo problemático e termina com sucesso** — a mesma que perdeu 10 shards em silêncio nas variantes. Os logs que registrariam descarte estão com 0 bytes.

MÉTODO Refazer o merge em modo estrito, que aborta em vez de engolir, após remover o pacote `com.sun.proxy` (classes geradas em runtime, nomeadas por JVM, que colidem entre shards).

RESULTADO **Zero descartes.** 12 dos 48 arquivos tinham o pacote, mas naquele run não colidiram. Os números publicados foram calculados sobre 24 shards em cada braço.

ACHADO EXTRA Ver abaixo — na contagem restrita, o sinal inverte em métodos alcançados.

5.1 · A inversão na contagem de métodos

Métodos alcançados: como a contagem muda o vencedor



A contagem global soma **todas** as classes do relatório, inclusive as classes de teste geradas — e esse universo difere entre braços: baseline 4.016 classes, Direct 4.165, WIT 4.163. Dividir numeradores de universos distintos produz números incomparáveis. Restrito aos métodos-alvo, com base idêntica, o WIT alcança mais.

MÉTRICA (EXPERIMENTO PRINCIPAL)	DIRECT	WIT	RAZÃO	VENCE
métodos alcançados	1.831	1.850	**1,0104**	**WIT**
blocks	9.610	9.434	0,9817	**Direct**
branches	6.156	6.013	0,9768	**Direct**
lines	14.249	14.215	0,9976	**Direct**

Cautela ao reportar: a inversão é de 1,0104× — 19 métodos em 1.831. A formulação defensável é *"o WIT alcança ao menos tantos métodos-alvo quanto o Direct, e a vantagem global aparente do Direct é artefato da contagem"*, não *"o WIT vence alcance"* com ênfase. Em branches e blocks o Direct vence nas duas contagens.

---

6 · Os defeitos de infraestrutura

Quatro, todos com o mesmo padrão: **falhavam parecendo sucesso**. Vale apresentá-los — mostram que a medição foi auditada, e cada um é uma ameaça à validade que outros trabalhos da área provavelmente têm sem saber.

DEFEITO	COMO SE MANIFESTAVA	EFEITO	ESTADO
Merge descartando em silêncio	terminava com sucesso; o relatório parecia normal	10 shards perdidos nas variantes; braços de tamanhos diferentes	**corrigido**
Driver sem repetição em falha	todos os builds reportavam sucesso	2 shards nunca executados	**corrigido**
Taxa de mutação não pareada	número plausível, calculado sobre bases diferentes	o baseline aparecia como −3,5pp; o valor correto é +0,5pp	**corrigido**
Compilador em US-ASCII	contado como "o teste não compila"	viés **sistemático contra o WIT**: até 95% dos descartes de um braço eram só comentário acentuado	**corrigido**

> **Por que o resultado principal sobreviveu a todos eles**
>
> Os três níveis de medição são independentes: o estático lê o código-fonte dos testes, o de execução lê os arquivos de resultado, o de cobertura mescla os relatórios. **O resultado principal — 1,33–1,76× de verificação forte — vem do nível estático**, que é determinístico e não passa por merge, execução nem compilação. Os quatro defeitos afetaram os outros dois níveis.

---

7 · Onde estão os dados

Tudo que sustenta os números deste documento está em dois lugares: o bucket S3, que guarda entradas e resultados de cada execução, e o repositório local, que guarda os testes gerados e as análises.

7.1 · S3 — execuções completas

Doze prefixos, um por execução. Cada um carrega **entradas e resultados juntos**, o que torna a reexecução e a auditoria triviais.

```
s3://witup-jcov-results/
│
├── paired-20260706-full02/    ← EXPERIMENTO PRINCIPAL (o do artigo)
│   ├── inputs/
│   │   ├── jcov-baseline.xml              60,6 MB  cobertura da suíte oficial
│   │   └── paired-shard-00..23.zip                 testes dos 2 braços
│   ├── paired-results/<braço>/shard-NN/
│   │   ├── jcov/jcov-result.xml           ~14 MB   cobertura da fatia
│   │   ├── jtr.tgz                                 resultado por teste
│   │   └── input-SHA256SUMS                        integridade da entrada
│   ├── final/                                      merges e relatórios
│   └── provenance/paired-merge-buildspec.yml       como o merge foi invocado
│
├── jdk-v1-20261002/     V1 · instrução de oráculo    650 objetos · 879 MB
├── jdk-v2-20261002/     V2 · modelo mini             650 objetos · 885 MB
├── jdk-v4-20261004/     V4 · few-shot                651 objetos · 883 MB
├── jdk-v3a-20261004/    V3a · controle (T=0,7)       649 objetos · 865 MB
├── reteste-v4/          8 shards reexecutados (variância de execução)
│
├── paired-20260928-jtr01/   reexecução que colheu os .jtr → revelou o oráculo mole
├── baseline-recheck-20260707/
└── pilot-*/                 pilotos de junho
```

Atenção a uma inconsistência de nomenclatura que já custou tempo: no experimento principal os diretórios são `shard-NN/`; nas variantes são `NN/`.

7.2 · Local — testes gerados e análises

CAMINHO	TAMANHO	CONTEÚDO
`~/Documents/tcc/runs/paired-20260706-full02/`	243 MB	dados do experimento principal: `final/`, `analysis/summary.json`, `pair-level.csv`
`~/Documents/tcc/tools/`	104 KB	scripts de análise do artigo: `throw_site_coverage.py`, `expath_type_match.py`
`~/Documents/tcc/HANDOFF_ESTADO_EXPERIMENTO.md`	12 KB	handoff do estado do experimento principal
`generated/experiments/jdk-variantes/jdk-v*/`	~600 MB cada	testes gerados e materializados de cada variante, com `pairing-manifest.csv`
`rerun-codebuild/mutacao*/`	~65 MB cada	mutação por condição: `alvos.csv`, `testes/`, `resultado.csv`
`rerun-codebuild/merge-*/out/`	~0,9–1,3 GB	XMLs mesclados e relatórios RepGen por condição
`sast2026-artifact/`	294 MB	pacote de artefato para submissão

7.3 · Arquivos-chave para conferir um número

PARA VERIFICAR…	OLHE EM…
verificação forte que passa (1,33–1,76×)	`rerun-codebuild/cruzar-variante.py` + os `.jtr` da variante
mira e voz por condição	mesma saída do `cruzar-variante.py`, tabela forte/frágil/outro
cobertura restrita	`rerun-codebuild/cobertura-restrita.py` + `manifest.csv` + relatórios RepGen
mutação (1,41× agregado)	`rerun-codebuild/mutacao*/resultado.csv`, colunas `site, arm, veredito`
variância de execução (726/726)	`rerun-codebuild/comparar-reteste.py` sobre `reteste/exec1` e `exec2`
o resultado antes da correção de encoding	`rerun-codebuild/mutacao*/resultado-sem-utf8.csv` (preservado de propósito)

---

8 · Como reexecutar

Cada estágio é independente e reentrante: pode ser repetido isoladamente a partir dos dados do estágio anterior. Os comandos abaixo são os que de fato produziram os números.

> **Pré-requisitos**
>
> Docker com a imagem `cloudarchlab/witup-evaluator:amd64` (traz JDK 11 de teste, jtreg e JCov 3.0); AWS CLI com acesso ao bucket e ao CodeBuild; Python 3.9+; chave da OpenAI apenas para o estágio 1. Em Apple Silicon a imagem roda emulada — use `--platform linux/amd64`, e conte com algo em torno de 35 minutos por variante na mutação.

Estágio 1 · Geração dos testes

Cada variante é definida por três variáveis de ambiente e um arquivo de configuração. É isto, e só isto, que muda entre os cenários:

```
# gerar-variantes.sh — assinatura: gerar <chave> <golden> <oracle> <config>

gerar jdk-v1  0 1 v1-pilot.runtime.json   # instrução de oráculo
gerar jdk-v2  0 0 v2.runtime.json         # modelo mini
gerar jdk-v4  1 0 v1-pilot.runtime.json   # few-shot (modelo obrigatório)
gerar jdk-v3a 0 0 v3.runtime.json         # controle, temperatura 0,7

# o que cada flag liga, em internal/aplicacao/prompts.go:
#   WITUP_PROMPT_GOLDEN=1  → REGRA DE OURO + MODELO OBRIGATÓRIO DE CABEÇALHO
#   WITUP_PROMPT_ORACLE=1  → bloco ORÁCULO OBRIGATÓRIO
```

```
bash gerar-variantes.sh          # submete os lotes à Batch API da OpenAI
bash daemon-submissao.sh         # acompanha e colhe as respostas
```

Os modelos e temperaturas ficam em `generated/configs/*.json`: `v1-pilot` e `v3` usam `gpt-4.1-nano` (T=0 e T=0,7); `v2` usa `gpt-4.1-mini` (T=0).

Estágio 2 · Execução

```
bash executar-variante.sh jdk-v1
#  1. materializa as respostas em .java pareados por método
#  2. divide em 24 shards por braço
#  3. envia para s3://witup-jcov-results/<stamp>/inputs/
#  4. dispara 48 builds no CodeBuild (jtreg + agente JCov)

bash varredura-shards.sh         # confere buracos e relança o que faltou
```

> **Não use `-boe skip` no merge**
>
> Essa opção descarta arquivo problemático e **termina com sucesso**. Foi assim que 10 shards sumiram em silêncio nas variantes. O `merge-jcov-estrito.sh` já a removeu — se houver conflito, ele falha, que é o comportamento desejado.

Estágio 3 · Merge da cobertura

```
cd rerun-codebuild

python3 tirar-proxy.py 'merge-jdk-v1/direct-tests/*.xml' \
                       'merge-jdk-v1/wit-context/*.xml'
#   remove com.sun.proxy: classes geradas em runtime, nomeadas $ProxyN
#   sequencialmente por JVM — o mesmo nome carrega estruturas diferentes
#   entre shards, e é isso que faz o Merger abortar

bash merge-jcov-estrito.sh jdk-v1 jdk-v2 jdk-v4 jdk-v3a

# experimento principal (layout shard-NN/, por isso script próprio):
bash remerge-original.sh
```

Estágio 4 · Análises

```
# classificação estática — produz o resultado principal
python3 cruzar-variante.py jdk-v1 "V1 instrução"

# cobertura restrita aos métodos-alvo, denominador por união
python3 cobertura-restrita.py manifest.csv \
    merge-jdk-v1/out/rep-baseline \
    merge-jdk-v1/out/rep-direct-tests \
    merge-jdk-v1/out/rep-wit-context

# robustez do classificador: mesma análise com padrão mais permissivo
python3 cruzar-variante-regex2.py jdk-v1 "V1 regex ampliado"
```

Estágio 5 · Mutação dirigida

```
python3 preparar-mutacao.py jdk-v1          # monta o diretório da condição
python3 preparar-mutacao.py jdk-v4 mutacao-v4b   # 2º arg: destino alternativo
#   o join é pelo pairing-manifest.csv — o hash do arquivo de teste deriva do
#   pair_id, que INCLUI a chave da variante, então hashes do baseline não casam

docker run --rm --platform linux/amd64 -m 4g \
  -v "$PWD/mutacao-v1:/m" cloudarchlab/witup-evaluator:amd64 \
  bash /m/rodar.sh

bash remutar.sh    # refaz as cinco condições com -encoding UTF-8
```

> **Dois erros a não repetir na mutação**
>
> **Encoding:** sem `-encoding UTF-8` no `javac`, todo teste com acento em comentário é contado como "não compila". O braço WIT escreve acento muito mais — era viés sistemático contra ele.
>
> **Pareamento:** a taxa só é comparável nos sites em que **os dois braços** têm veredito válido. Comparar sobre conjuntos diferentes inverteu o sinal do experimento principal, de +0,5pp para −3,5pp.

Estágio 6 · Variância de execução

```
# relança 8 shards já executados num prefixo separado, preservando o original
aws s3 cp s3://witup-jcov-results/jdk-v4-20261004/inputs/ \
          s3://witup-jcov-results/reteste-v4/inputs/ --recursive

# depois de concluídos, compara teste a teste
python3 comparar-reteste.py reteste/exec1 reteste/exec2
```

Ordem mínima para reproduzir o resultado principal

Se o objetivo for apenas confirmar os 1,33–1,76×, **não é preciso reexecutar nada na nuvem**. A classificação estática lê o código-fonte dos testes já materializados e os arquivos de resultado já baixados:

```
cd rerun-codebuild
for K in jdk-v1 jdk-v2 jdk-v4 jdk-v3a; do
    python3 cruzar-variante.py $K "$K"
done
```

Isso roda em segundos e é determinístico — a mesma entrada produz saída idêntica bit a bit, verificado por md5 em três execuções. É também a razão de o resultado principal ter sobrevivido aos quatro defeitos de infraestrutura: ele não passa por merge, execução nem compilação.

Custos observados

ESTÁGIO	CUSTO	OBSERVAÇÃO
Geração (4 variantes)	`crédito OpenAI`	V3b e V3c pararam no limite de cobrança
Execução (96 shards)	`US$ 39`	CodeBuild MEDIUM; mediana de 33 min por shard
Reteste (8 shards)	`US$ 2,64`	~US$ 0,33 por shard
Merge e mutação	`local`	~45 min por merge, ~35 min por condição de mutação (emulado)

---

9 · O limite do estudo

> **Conformidade de contrato, não detecção de defeito real**
>
> O WIT lê o código e extrai *"este método lança NPE quando o argumento é null"*. O teste verifica exatamente isso. Mas se o código tivesse um bug e lançasse a exceção errada, o WIT extrairia a exceção errada, o teste verificaria a exceção errada — e **passaria**.
>
> É como conferir se um relógio está certo olhando para o próprio relógio. Quebrar o círculo exige uma fonte de verdade independente do código: mutação dirigida (sintética — feita, seção 4.4) ou replay de bugs reais (Defects4J — artigo à parte).

Outras ressalvas em aberto

- **Sem barra de erro.** V3b e V3c nunca foram gerados; a variância entre amostras de geração segue sem estimativa.
- **Granularidade da cobertura.** Medida em (classe, nome do método), com sobrecargas agregadas. A regra é a mesma nos três braços, então a comparação é justa, mas é mais grossa que por assinatura.
- **832 dos métodos-alvo nunca foram carregados** por nenhum braço nem pelo baseline, e ficam fora de qualquer medida.
- **O few-shot do V4 é ambíguo** — foi escrito para fixar o formato do cabeçalho, mas também demonstra um oráculo forte.

10 · Próximos passos

PASSO	CUSTO	O QUE DESTRAVA
Operador de mutação por tipo errado	~1 dia	mutante que só morre se o teste verifica o **tipo** — onde o WIT tem 1,3–1,6×. É o passo que mais agrega.
Regerar V3b e V3c	billing OpenAI	a barra de erro dos 1,33–1,76×
Ampliar a amostra de variância	~US$ 0,33/shard	estender a afirmação de reprodutibilidade além dos 726 testes
Replay de bugs reais (Defects4J)	artigo à parte	detecção com falha real, não sintética — fecha a crítica central